```
"""
DCOMP320 - Data Structures and Algorithms
Assignment: Stacks Implementation (Version
2 - Postfix approach)

Approach:
  1. Convert each infix expression to
postfix (Reverse Polish Notation)
     using a stack (the Shunting-Yard
algorithm).
  2. Evaluate the postfix expression using
a second stack.
Reads expressions from input.txt and writes
results to output.txt.
"""

import sys


# ----------------------------------------
----------------------------
# Stack implemented with a singly linked
list
# ----------------------------------------
----------------------------
class Node:
    def __init__(self, data,
next_node=None):
        self.data = data
        self.next = next_node
```

```
class Stack:
    """LIFO stack built from linked nodes.
The top of the stack is the head."""

    def __init__(self):
        self._top = None
        self._size = 0

    def push(self, item):
        self._top = Node(item, self._top)
        self._size += 1

    def pop(self):
        if self._top is None:
            raise IndexError("pop from
empty stack")
        item = self._top.data
        self._top = self._top.next
        self._size -= 1
        return item

    def peek(self):
        if self._top is None:
            raise IndexError("peek at empty
stack")
        return self._top.data

    def is_empty(self):
        return self._top is None

    def size(self):
        return self._size
```

```
# ----------------------------------------
----------------------------
# Expression handling
# ----------------------------------------
----------------------------
PRECEDENCE = {"+": 1, "-": 1, "*": 2, "/":
2, "u-": 3}   # u- = unary minus
RIGHT_ASSOC = {"u-"}
BINARY = {"+", "-", "*", "/"}


def tokenize(text):
    """Turn a string into a list of tokens
(numbers, operators, parentheses).
    A minus sign in a unary position is
turned into the token 'u-'."""
    tokens, i = [], 0
    while i < len(text):
        ch = text[i]
        if ch.isspace():
            i += 1
        elif ch.isdigit() or ch == ".":
            j = i
            while j < len(text) and
(text[j].isdigit() or text[j] == "."):
                j += 1
            number = text[i:j]
            if number.count(".") > 1 or
number == ".":
                raise ValueError(f"invalid
number '{number}'")
            tokens.append(float(number))
```

```
            i = j
        elif ch in "+-*/()":
            previous = tokens[-1] if tokens
else None
            unary = previous is None or
previous == "(" or previous in PRECEDENCE
            if ch == "-" and unary:
                tokens.append("u-")
            elif ch == "+" and unary:
                pass
# unary plus changes nothing
            else:
                tokens.append(ch)
            i += 1
        else:
            raise ValueError(f"invalid
character '{ch}'")
    return tokens


def to_postfix(tokens):
    """Shunting-Yard: convert infix tokens
to a postfix list using a stack."""
    output, ops = [], Stack()
    for tok in tokens:
        if isinstance(tok, float):
            output.append(tok)
        elif tok == "(":
            ops.push(tok)
        elif tok == ")":
            while not ops.is_empty() and
ops.peek() != "(":
                output.append(ops.pop())
            if ops.is_empty():
```

```
                raise
ValueError("mismatched parentheses")
            ops.pop()
# remove '('
        else:
# an operator
            while (not ops.is_empty() and
ops.peek() != "("
                   and
(PRECEDENCE[ops.peek()] > PRECEDENCE[tok]
                        or
(PRECEDENCE[ops.peek()] == PRECEDENCE[tok]
                            and tok not in
RIGHT_ASSOC))):
                output.append(ops.pop())
            ops.push(tok)
    while not ops.is_empty():
        top = ops.pop()
        if top == "(":
            raise ValueError("mismatched
parentheses")
        output.append(top)
    return output


def evaluate_postfix(postfix):
    """Evaluate a postfix token list using
a stack of operands."""
    st = Stack()
    try:
        for tok in postfix:
            if isinstance(tok, float):
                st.push(tok)
            elif tok == "u-":
```

```
                st.push(-st.pop())
            else:
                b = st.pop()
                a = st.pop()
                if tok == "+":
                    st.push(a + b)
                elif tok == "-":
                    st.push(a - b)
                elif tok == "*":
                    st.push(a * b)
                elif tok == "/":
                    if b == 0:
                        raise
ZeroDivisionError("division by zero")
                    st.push(a / b)
        result = st.pop()
    except IndexError:
        raise ValueError("invalid
expression")
    if not st.is_empty():
        raise ValueError("invalid
expression")
    return result


def evaluate(expression):
    return
evaluate_postfix(to_postfix(tokenize(expres
sion)))


# ----------------------------------------
----------------------------
# File handling
```

```
# ----------------------------------------
----------------------------
def format_result(value):
    if value == int(value):
        return str(int(value))
    return ("%.6f" %
value).rstrip("0").rstrip(".")


def is_separator(line):
    s = line.strip()
    return len(s) >= 3 and set(s) == {"-"}


def process_file(input_path, output_path):
    try:
        with open(input_path, "r") as f:
            lines = f.read().splitlines()
    except FileNotFoundError:
        print(f"Error: cannot find
'{input_path}'")
        return None

    out = []
    for line in lines:
        if is_separator(line):
            out.append(line.strip())
        elif line.strip():
            try:

out.append(format_result(evaluate(line)))
            except (ValueError,
ZeroDivisionError) as err:
                out.append(f"Error: {err}")
```

```
    with open(output_path, "w") as f:
        f.write("\n".join(out) + "\n")
    return out


def main():
    in_file = sys.argv[1] if len(sys.argv)
> 1 else "input.txt"
    out_file = sys.argv[2] if len(sys.argv)
> 2 else "output.txt"
    if process_file(in_file, out_file) is
not None:
        print(f"Results written to
{out_file}")


if __name__ == "__main__":
    main()
```